CHAPTER

7 Spin locks and contention

When writing programs for uniprocessors, it is usually safe to ignore the underlying system's architectural details. Unfortunately, multiprocessor programming has yet to reach that state; for now, it is crucial to understand the underlying machine architecture. The goal of this chapter is to explain how architecture affects performance, and how to exploit this knowledge to write efficient concurrent programs. We revisit the familiar mutual exclusion problem, this time with the aim of devising mutual exclusion protocols that work well with today's multiprocessors.

Any mutual exclusion protocol poses the question: "What do you do if you cannot acquire the lock?" There are two alternatives. If you keep trying, the lock is called a *spin lock*, and repeatedly testing the lock is called *spinning*, or *busy-waiting*. The `Filter` and `Bakery` algorithms are spin locks. Spinning makes sense when you expect the lock delay to be short (and only on multiprocessors, of course). The alternative is to suspend yourself and ask the operating system to schedule another thread on your processor, which is sometimes called *blocking*. Because switching from one thread to another is expensive, blocking makes sense only if you expect the lock delay to be long. Many operating systems mix both strategies, spinning for a short time and then blocking. Both spinning and blocking are important techniques. In this chapter, we turn our attention to locks that use spinning.

7.1 Welcome to the real world

We approach real-world mutual exclusion via the `Lock` interface from the `java.util.concurrent.locks` package. For now, we consider only the two principal methods, `lock()` and `unlock()`. As mentioned in Pragma 2.2.1, these methods are often used in the following structured way:

```
1 Lock mutex = new LockImpl(...); // lock implementation
2 ...
3 mutex.lock();
4 try {
5   ...          // body
6 } finally {
7   ... // restore object invariant if needed
8   mutex.unlock();
9 }
```

The Art of Multiprocessor Programming. https://doi.org/10.1016/B978-0-12-415950-1.00017-3

We create a new Lock object called mutex (line 1). Because Lock is an interface and not a class, we cannot create Lock objects directly. Instead, we create an object that *implements* the Lock interface. (The java.util.concurrent.locks package includes several classes that implement Lock, and we provide others in this chapter.) Next, we acquire the lock (line 3), and enter the critical section in a **try** block (line 4). The **finally** block (line 6) ensures that no matter what, the lock is released when control leaves the critical section. We do not put the lock() call inside the **try** block, because the lock() call might throw an exception before acquiring the lock, causing the **finally** block to call unlock() when the lock has not actually been acquired. (Java does not permit instructions to be executed between program lines, so once line 3 is completed and the lock is taken, the thread is in the **try** block.)

Why not use one of the Lock algorithms studied in Chapter 2, such as Filter or Bakery? One reason is the space lower bound proved in Chapter 2: No matter what we do, mutual exclusion using reads and writes requires space linear in n, the number of threads that potentially access the location. It gets worse.

Consider, for example, the two-thread Peterson lock algorithm of Chapter 2, presented again in Fig. 7.1. There are two threads, with IDs 0 and 1. When thread A wants to acquire the lock, it sets flag[A] to *true*, sets victim to A, and tests victim and flag[$1 - A$]. As long as victim is A and flag[$1 - A$] is *true*, the thread spins, repeating the test. Once either victim is not A or flag[$1 - A$] is *false*, the thread enters the critical section, setting flag[A] to *false* as it leaves. We know from Chapter 2 that the Peterson lock provides starvation-free mutual exclusion.

Suppose we write a simple concurrent program in which each of the threads repeatedly acquires the Peterson lock, increments a shared counter, and then releases the lock. We run it on a multiprocessor, where each thread executes this acquire–increment–release cycle, say, half a million times. On most modern architectures, the threads finish quickly. Alarmingly, however, we may discover that the counter's final value may be slightly off from the expected million mark. Proportionally, the error

```
1  class Peterson implements Lock {
2    private boolean[] flag = new boolean[2];
3    private int victim;
4    public void lock() {
5      int i = ThreadID.get(); // either 0 or 1
6      int j = 1-i;
7      flag[i] = true;
8      victim = i;
9      while (flag[j] && victim == i) {}; // spin
10   }
11 }
```

FIGURE 7.1

The Peterson class (Chapter 2): the order of reads and writes in lines 7–9 is crucial to providing mutual exclusion.

may be tiny, but why is there any error at all? Somehow, it must be that both threads are occasionally in the critical section at the same time, even though we have proved that this cannot happen. To quote Sherlock Holmes:

> *How often have I said to you that when you have eliminated the impossible, whatever remains, however improbable, must be the truth?*

Our proof fails, not because there is anything wrong with our logic, but because our assumptions about the real world are mistaken.

When programming our multiprocessor, we implicitly assumed that read–write operations are atomic, that is, they are linearizable to some sequential execution, or at the very least, that they are sequentially consistent. (Recall that linearizability implies sequential consistency.) As we saw in Chapter 3, sequential consistency implies that there is some global order on all operations in which each thread's operations take effect as ordered by its program. When we proved the `Peterson` lock correct, we relied, without calling attention to it, on the assumption that memory is sequentially consistent. In particular, mutual exclusion depends on the order of the steps in lines 7–9 of Fig. 7.1. Our proof that the `Peterson` lock provides mutual exclusion implicitly relied on the assumption that any two memory accesses by the same thread, even to separate variables, take effect in program order. Specifically, B's write to `flag`$[B]$ must take effect before its write to `victim` (Eq. (2.3.2)) and A's write to `victim` must take effect before its read of `flag`$[B]$ (Eq. (2.3.4)).

Unfortunately, modern multiprocessors, and programming languages for modern multiprocessors, typically do not provide sequentially consistent memory, nor do they necessarily guarantee program order among reads and writes by a given thread.

Why not? The first culprits are compilers that reorder instructions to enhance performance. It is possible that the order of writes by thread B of `flag`$[B]$ and `victim` will be reversed by the compiler, invalidating Eq. (2.3.2). In addition, if a thread reads a variable repeatedly without writing it, a compiler may eliminate all but the first read of the variable, using the value read the first time for all subsequent reads. For example, the loop on line 9 of Fig. 7.1 may be replaced with a conditional statement that spins forever if the thread may not immediately enter the critical section.

A second culprit is the multiprocessor hardware itself. (Appendix B has a more extensive discussion of the multiprocessor architecture issues raised in this chapter.) Hardware vendors make no secret of the fact that writes to multiprocessor memory do not necessarily take effect when they are issued, because in most programs the vast majority of writes do not *need* to take effect in shared memory right away. On many multiprocessor architectures, writes to shared memory are buffered in a special *write buffer* (sometimes called a *store buffer*), to be written to memory only when needed. If thread A's write to `victim` is delayed in a write buffer, it may arrive in memory only after A reads `flag`$[B]$, invalidating Eq. (2.3.4).

How then does one program multiprocessors, given such weak memory consistency guarantees? To prevent the reordering of operations resulting from write buffering, modern architectures provide a special *memory barrier* instruction (sometimes called a *memory fence*) that forces outstanding operations to take effect. Synchroniza-

tion methods such as `getAndSet()` and `compareAndSet()` of `AtomicInteger`, include a memory barrier on many architectures, as do `reads` and `writes` to **`volatile`** fields. It is the programmer's responsibility to know where memory barriers are needed (e.g., the `Peterson` lock can be fixed by placing a barrier immediately before each read, and how to insert them. We discuss how to do this for Java in the next section.

Not surprisingly, memory barriers are expensive, so we want to minimize their use. Because operations such as `getAndSet()` and `compareAndSet()` have higher consensus numbers than `reads` and `writes`, and can be used in a straightforward way to reach a kind of consensus on who can and cannot enter the critical section, it may be sensible to design mutual exclusion algorithms that use these operations directly.

7.2 Volatile fields and atomic objects

As a rule of thumb, any object field, accessed by concurrent threads, that is not protected by a critical section should be declared **`volatile`**. Without such a declaration, that field will not act like an atomic register: Reads may return stale values, and writes may be delayed.

A **`volatile`** declaration does not make compound operations atomic: If x is a volatile variable, then the expression x++ will not necessarily increment x if concurrent threads can modify x. For tasks such as these, the java.util.concurrent.atomic package provides classes such as `AtomicReference<T>` or `AtomicInteger` that provide many useful atomic operations.

In earlier chapters, we did not put **`volatile`** declarations in our pseudocode because we assumed memory was linearizable. From now on, however, we assume the Java memory model, and so we put **`volatile`** declarations where they are needed. The Java memory model is described in more detail in Appendix A.3.

7.3 Test-and-set locks

The principal synchronization instruction on many early multiprocessor architectures was the *test-and-set* instruction. It operates on a single memory word (or byte) that may be either *true* or *false*. The test-and-set instruction, which has consensus number two, atomically stores *true* in the word and returns that word's previous value; that is, it *swaps* the value *true* for the word's current value. At first glance, test-and-set seems ideal for implementing a spin lock: The lock is free when the word's value is *false*, and busy when it is *true*. The `lock()` method repeatedly applies test-and-set to the word until it returns *false* (i.e., until the lock is free). The `unlock()` method simply writes the value *false* to it.

The `TASLock` class in Fig. 7.2 implements this lock in Java using the `AtomicBoolean` class in the java.util.concurrent package. This class stores a Boolean value, and it provides a `set(`b`)` method to replace the stored value with value b, and a

```
1  public class TASLock implements Lock {
2    AtomicBoolean state = new AtomicBoolean(false);
3    public void lock() {
4      while (state.getAndSet(true)) {}
5    }
6    public void unlock() {
7      state.set(false);
8    }
9  }
```

FIGURE 7.2

The TASLock class.

```
1  public class TTASLock implements Lock {
2    AtomicBoolean state = new AtomicBoolean(false);
3    public void lock() {
4      while (true) {
5        while (state.get()) {};
6        if (!state.getAndSet(true))
7          return;
8      }
9    }
10   public void unlock() {
11     state.set(false);
12   }
13 }
```

FIGURE 7.3

The TTASLock class.

getAndSet(b) that atomically replaces the current value with b and returns the previous value. The test-and-set instruction is equivalent to getAndSet($true$). (We follow common practice by using test-and-set in prose, but we use getAndSet(*true*) in our code examples to be compatible with Java.)

Now consider TTASLock (Fig. 7.3), a variant of the TASLock algorithm called a *test-and-test-and-set* lock. In this algorithm, a thread reads the lock to check that it is free *before* performing the test-and-set. If the lock is not free, the thread repeatedly reads the lock until it is (i.e., until get() returns *false*), and only after that does the thread apply test-and-set. From the point of view of correctness, TASLock and TTASLock are equivalent: Each guarantees deadlock-free mutual exclusion. Under the simple model we have been using so far, there should be no difference between these two algorithms.

How do they compare on a real multiprocessor? Experiments that measure the elapsed time for n threads to execute a short critical section a fixed total number of times invariably yield results that look like Fig. 7.4. Each data point represents the

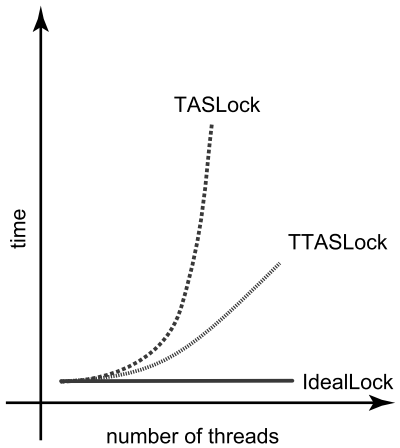

FIGURE 7.4

Schematic performance of a `TASLock`, a `TTASLock`, and an ideal lock with no overhead.

same amount of work, so in the absence of contention effects, all curves would be flat. The top curve is the `TASLock`, the middle curve is the `TTASLock`, and the bottom curve shows the time that would be needed if the threads did not interfere at all. The difference is dramatic: The `TASLock` performs very poorly; the `TTASLock` performance, while substantially better, still falls far short of the ideal.

To understand these results, we must study the architecture of modern multiprocessors. First, a word of caution: Modern multiprocessors have a variety of architectures, so we must be careful about overgeneralizing. Nevertheless, (almost) all modern architectures have similar issues concerning caching and locality. The details differ, but the principles remain the same.

For simplicity, we consider a typical multiprocessor architecture in which processors communicate by a shared broadcast medium called a *bus*. The memory typically also resides in nodes connected to the bus, each with its own *memory controller*. The processors and memory controllers can broadcast on the bus, but only one at a time. All processors and memory controllers can listen at the same time. Bus-based architectures are common today because they are easy to build, but they do not scale well to many processors; the bus becomes a point of contention.

Each processor has a *cache*, a small high-speed memory in which it keeps data likely to be of interest. Memory access typically takes orders of magnitude longer than access to the cache. Technology trends are not helping: Memory access time is unlikely to catch up with processor cycle time in the near future, so cache performance is critical to the overall performance of a multiprocessor.

A processor's cache contains copies of memory locations, along with their addresses. These copies are maintained by a *cache coherence protocol*, and may be *shared* or *exclusive*. As the name suggests, if any processor has an exclusive copy

of a memory location, then no other processor has a copy of that location, shared or exclusive.

When accessing a memory location, a processor first checks whether its cache has a copy of that location. If it is writing the location, the copy must be exclusive. If the cache has the location's current data, then we say that the processor *hits* in its cache. In this case, the processor may read or write the copy in its cache immediately. Otherwise, the processor has a *cache miss*, and it requests a copy by broadcasting the address of the location on the bus. The other processors (and the memory controllers) *snoop* on the bus. If some processor has an exclusive copy of the location in its cache, it responds by broadcasting the address and value (making its copy shared). Otherwise, the memory controller responsible for that location responds. If the request was to write the location, then all previous copies are *invalidated*, so that the requester has an exclusive copy of that location.

We now consider how the simple `TASLock` algorithm performs on this architecture: Because `getAndSet()` may write the location, a thread must request an exclusive copy of the lock whenever it calls `getAndSet()`, unless its processor's cache already has such a copy. This request forces other processors to invalidate their cached copies of the lock. If multiple threads are spinning on the lock, almost every call to `getAndSet()` will result in a cache miss and a request on the bus to fetch the (unchanged) value. Compounding the injury, when the thread holding the lock tries to release it, it may be delayed because the bus is monopolized by the spinners. Indeed, because all threads use the bus to communicate with memory, even threads not waiting for the lock may be delayed. We now understand why the `TASLock` performs so poorly.

Now consider the behavior of the `TTASLock` algorithm while the lock is held by a thread A. The first time thread B reads the lock, it has a cache miss, forcing B to block while the value is loaded into B's cache. However, because B is only reading the lock, it only requests a shared copy, which is stored in its processor's cache. As long as A holds the lock, B repeatedly rereads the value, but hits in its cache every time. B produces no bus traffic after its first request, and does not slow down other threads' memory accesses.

The situation deteriorates, however, when the lock holder A releases the lock by writing *false* to the lock's `state` variable. Because the lock is now shared with all the threads spinning on it, this write causes a cache miss, resulting in a request on the bus for an exclusive copy of the lock. This request invalidates the cached copies of the spinning threads. Each one has a cache miss and rereads the new value, and they all (more or less simultaneously) call `getAndSet()` to acquire the lock. The first to succeed invalidates the others, which must then reread the value, causing a storm of bus traffic. Eventually, the threads settle down once again to local spinning.

This notion of *local spinning*, where threads repeatedly reread cached values instead of repeatedly using the bus, is an important principle critical to the design of efficient spin locks.

7.4 Exponential back-off

We now consider how to improve the TTASLock algorithm by reducing the bus traffic induced when a thread releases the lock and many threads are waiting to acquire it. First, some terminology: *Contention* on a lock occurs when multiple threads try to acquire the lock at the same time. *High contention* means there are many such threads; *low contention* means there are few. As discussed above, attempting to acquire a highly contended lock is a bad idea: Such an attempt contributes to bus traffic (making the traffic jam worse) at a time when the thread's chances of acquiring the lock are slim. Instead, it is more effective for the thread to *back off* for some duration, giving the competing threads a chance to finish.

Recall that in the TTASLock class, the lock() method takes two steps: It repeatedly reads the lock until the lock is free, and then it attempts to acquire the lock by calling getAndSet(*true*). Here is a key observation: If a thread fails to acquire the lock in the second step, then some other thread must have acquired the lock between the first and second step, so most likely there is high contention for that lock. Here is a simple approach: Whenever a thread sees the lock has become free but fails to acquire it, it backs off before retrying. To ensure that competing threads do not fall into lockstep, each backing off and then trying again to acquire the lock at the same time, the thread backs off for a random duration.

For how long should the thread back off before retrying? A good rule of thumb is that the larger the number of unsuccessful tries, the higher the likely contention, so the longer the thread should back off. To incorporate this rule, each time a thread tries and fails to get the lock, it doubles the expected back-off time, up to a fixed maximum.

Because backing off is common to several locking algorithms, we encapsulate this logic in a simple Backoff class, shown in Fig. 7.5. The constructor takes two arguments: minDelay is the initial minimum delay (it makes no sense for the thread to back off for too short a duration), and maxDelay is the final maximum delay (a final limit is necessary to prevent unlucky threads from backing off for much too long). The limit field controls the current delay limit. The backoff() method computes a random delay between zero and the current limit, and blocks the thread for that duration before returning. It doubles the limit for the next back-off, up to maxDelay.

Fig. 7.6 shows the BackoffLock class. It uses a Backoff object whose minimum and maximum back-off durations are governed by the constants chosen for MIN_DELAY and MAX_DELAY. Note that the thread backs off only when it fails to acquire a lock that it had immediately before observed to be free. Observing that the lock is held by another thread says nothing about the level of contention.

The BackoffLock is easy to implement, and typically performs significantly better than TASLock and TTASLock on many architectures. Unfortunately, its performance is sensitive to the choice of MIN_DELAY and MAX_DELAY values. To deploy this lock on a particular architecture, it is easy to experiment with different values, and to choose the ones that work best. Experience shows, however, that these optimal values are sensitive to the number of processors and their speed, so it is not easy to tune BackoffLock to be portable across a range of different machines.

```
1  public class Backoff {
2    final int minDelay, maxDelay;
3    int limit;
4    public Backoff(int min, int max) {
5      minDelay = min;
6      maxDelay = max;
7      limit = minDelay;
8    }
9    public void backoff() throws InterruptedException {
10     int delay = ThreadLocalRandom.current().nextInt(limit);
11     limit = Math.min(maxDelay, 2 * limit);
12     Thread.sleep(delay);
13   }
14 }
```

FIGURE 7.5

The `Backoff` class: adaptive back-off logic. To ensure that concurrently contending threads do not repeatedly try to acquire the lock at the same time, threads back off for a random duration. Each time the thread tries and fails to get the lock, it doubles the expected time to back off, up to a fixed maximum.

```
1  public class BackoffLock implements Lock {
2    private AtomicBoolean state = new AtomicBoolean(false);
3    private static final int MIN_DELAY = ...;
4    private static final int MAX_DELAY = ...;
5    public void lock() {
6      Backoff backoff = new Backoff(MIN_DELAY, MAX_DELAY);
7      while (true) {
8        while (state.get()) {};
9        if (!state.getAndSet(true)) {
10         return;
11       } else {
12         backoff.backoff();
13       }
14     }
15   }
16   public void unlock() {
17     state.set(false);
18   }
19   ...
20 }
```

FIGURE 7.6

The exponential back-off lock. Whenever the thread fails to acquire a lock that became free, it backs off before retrying.

One drawback of BackoffLock is that it underutilizes the critical section when the lock is contended: Because threads back off when they notice contention, when a thread releases the lock, there may be some delay before another thread attempts to acquire it, even though many threads are waiting to acquire the lock. Indeed, because threads back off for longer at higher contention, this effect is more pronounced at higher levels of contention.

Finally, the BackoffLock can be unfair, allowing one thread to acquire the lock many times while other threads are waiting. TASLock and TTASLock may also be unfair, but BackoffLock exacerbates this problem because the thread that just released the lock might never notice that the lock is contended, and so not back off at all.

Although this unfairness has obvious negative consequences, including the possibility of starving other threads, it also has some positive consequences: Because a lock often protects accesses to some shared data structure, which is also cached, granting repeated access to the same thread without intervening accesses by threads at different processors reduces cache misses due to accesses to this data structure, and so reduces bus traffic and avoids the latency of communication. For longer critical sections, this effect can be more significant than the effect of reduced contention on the lock itself. So there is a tension between fairness and performance.

7.5 Queue locks

We now explore a different approach to implementing scalable spin locks, one that is slightly more complicated than back-off locks, but inherently more portable, and avoids or ameliorates many of the problems of back-off locks. The idea is to have threads waiting to acquire the lock form a *queue*. In a queue, each thread can discover when its turn has arrived by checking whether its predecessor has finished. Cache-coherence traffic is reduced by having each thread spin on a different location. A queue also allows better utilization of the critical section, since there is no need to guess when to attempt to access it: Each thread is notified directly by its predecessor in the queue. Finally, a queue provides first-come-first-served fairness, the same high degree of fairness achieved by the Bakery algorithm. We now explore different ways to implement *queue locks*, a family of locking algorithms that exploit these insights.

7.5.1 Array-based locks

Fig. 7.7 shows the ALock,[1] a simple array-based queue lock. The threads share an AtomicInteger tail field, initially zero. To acquire the lock, each thread atomically increments tail (line 17). Call the resulting value the thread's *slot*. The slot is used as an index into a Boolean flag array.

If flag[j] is *true*, then the thread with slot j has permission to acquire the lock. Initially, flag[0] is *true*. To acquire the lock, a thread spins until the flag at its slot

[1] Most of our lock classes use the initials of their inventors, as explained in Section 7.11.

```
1  public class ALock implements Lock {
2    ThreadLocal<Integer> mySlotIndex = new ThreadLocal<Integer> (){
3      protected Integer initialValue() {
4        return 0;
5      }
6    };
7    AtomicInteger tail;
8    volatile boolean[] flag;
9    int size;
10   public ALock(int capacity) {
11     size = capacity;
12     tail = new AtomicInteger(0);
13     flag = new boolean[capacity];
14     flag[0] = true;
15   }
16   public void lock() {
17     int slot = tail.getAndIncrement() % size;
18     mySlotIndex.set(slot);
19     while (!flag[slot]) {};
20   }
21   public void unlock() {
22     int slot = mySlotIndex.get();
23     flag[slot] = false;
24     flag[(slot + 1) % size] = true;
25   }
26 }
```

FIGURE 7.7

Array-based queue lock.

becomes *true* (line 19). To release the lock, the thread sets the flag at its slot to *false* (line 23), and sets the flag at the next slot to *true* (line 24). All arithmetic is modulo n, where n is at least as large as the maximum number of concurrent threads.

In the `ALock` algorithm, `mySlotIndex` is a *thread-local* variable (see Appendix A). Thread-local variables differ from their regular counterparts in that each thread has its own, independently initialized copy of each variable. Thread-local variables need not be stored in shared memory, do not require synchronization, and do not generate any coherence traffic since they are accessed by only one thread. The value of a thread-local variable is accessed by `get()` and `set()` methods.

The `flag[]` array, on the other hand, is shared.[2] However, contention on the array locations is minimized since each thread, at any given time, spins on its locally cached copy of a single array location, greatly reducing invalidation traffic.

[2] The role of the **`volatile`** declaration here is not to introduce a memory barrier but rather to prevent the compiler from applying any optimizations to the loop in line 19.

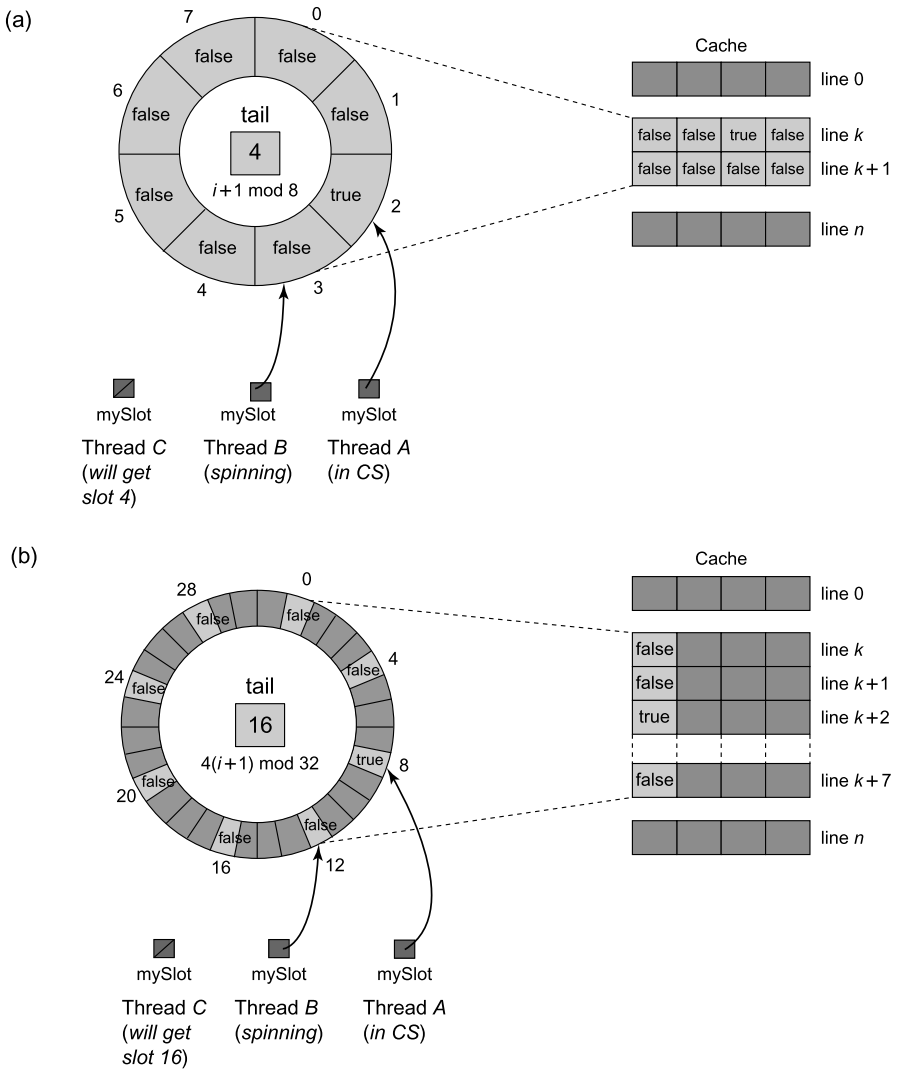

FIGURE 7.8

The `ALock` with padding to avoid false sharing. In part (a), the `ALock` has eight slots which are accessed via a modulo 8 counter. Array entries are typically mapped into cache lines consecutively. As illustrated, when thread *A* changes the status of its entry, thread *B*, whose entry is mapped to the same cache line *k*, incurs a false invalidation. In part (b), each location is padded so it is 4 apart from the others with a modulo 32 counter. Even if array entries are mapped consecutively, the entry for *B* is mapped to a different cache line from that of *A*, so *B*'s entry is not invalidated when *A* invalidates its entry.

Contention may still occur because of a phenomenon called *false sharing*, which occurs when adjacent data items (such as array elements) share a single cache line. A write to one item invalidates that item's cache line, which causes invalidation traffic to processors that are spinning on nearby unchanged items that happen to fall in the same cache line. In the example in part (a) of Fig. 7.8, threads accessing the eight `ALock` locations may suffer unnecessary invalidations because the locations were all cached in the same two four-word lines.

One way to avoid false sharing is to *pad* array elements so that distinct elements are mapped to distinct cache lines. Padding is easier in low-level languages like C or C++, where the programmer has direct control over the layout of objects in memory. In the example in part (b) of Fig. 7.8, we pad the eight original `ALock` locations by increasing the lock array size four-fold, and placing the locations four words apart so that no two locations can fall in the same cache line. (We increment from one location i to the next by computing $4(i + 1) \bmod 32$ instead of $i + 1 \bmod 8$.)

The `ALock` improves on `BackoffLock`: it reduces invalidations to a minimum and minimizes the interval between when a lock is freed by one thread and when it is acquired by another. Unlike the `TASLock` and `BackoffLock`, this algorithm guarantees that no starvation occurs, and provides first-come-first-served fairness.

Unfortunately, the `ALock` lock is not space-efficient. It requires a known bound n on the maximum number of concurrent threads, and it allocates an array of that size per lock. Synchronizing L distinct objects requires $O(Ln)$ space, even if a thread accesses only one lock at a time.

7.5.2 The CLH queue lock

We now turn our attention to a different style of queue lock, the `CLHLock` (Fig. 7.9). This class records each thread's status in a `QNode` object, which has a Boolean `locked` field. If that field is *true*, then the corresponding thread has either acquired the lock or is waiting for the lock. If that field is *false*, then the thread has released the lock. The lock itself is represented as a virtual linked list of `QNode` objects. We use the term "virtual" because the list is implicit: Each thread refers to its predecessor through a thread-local `pred` variable. The public `tail` field is an `AtomicReference<QNode>` to the node most recently added to the queue.

To acquire the lock, a thread sets the `locked` field of its `QNode` to *true*, indicating that the thread is not ready to release the lock. The thread applies `getAndSet()` to the `tail` field to make its own node the tail of the queue, simultaneously acquiring a reference to its predecessor's `QNode`. The thread then spins on the predecessor's `locked` field until the predecessor releases the lock. To release the lock, the thread sets its node's `locked` field to *false*. It then reuses its predecessor's `QNode` as its new node for future lock accesses. It can do so because at this point the thread's predecessor's `QNode` is no longer used by the predecessor. It cannot use its old `QNode` because that node could be referenced both by the thread's successor and by the `tail`. Although we do not do so in our implementation, it is possible to recycle nodes so that if there are L locks and each thread accesses at most one lock at a time, then the `CLHLock` class

```
1  public class CLHLock implements Lock {
2    AtomicReference<QNode> tail;
3    ThreadLocal<QNode> myPred;
4    ThreadLocal<QNode> myNode;
5    public CLHLock() {
6      tail = new AtomicReference<QNode>(new QNode());
7      myNode = new ThreadLocal<QNode>() {
8        protected QNode initialValue() {
9          return new QNode();
10       }
11     };
12     myPred = new ThreadLocal<QNode>() {
13       protected QNode initialValue() {
14         return null;
15       }
16     };
17   }
18   public void lock() {
19     QNode qnode = myNode.get();
20     qnode.locked = true;
21     QNode pred = tail.getAndSet(qnode);
22     myPred.set(pred);
23     while (pred.locked) {}
24   }
25   public void unlock() {
26     QNode qnode = myNode.get();
27     qnode.locked = false;
28     myNode.set(myPred.get());
29   }
30   class QNode {
31     volatile boolean locked = false;
32   }
33 }
```

FIGURE 7.9

The `CLHLock` class.

needs only $O(L + n)$ space, as compared with $O(Ln)$ for the `ALock` class.[3] Fig. 7.10 shows a typical `CLHLock` execution.

Like the `ALock`, this algorithm has each thread spin on a distinct location, so when one thread releases its lock, it invalidates only its successor's cache. This algorithm requires much less space than the `ALock` class, and does not require knowledge of

[3] There is no need to reuse nodes in garbage-collected languages such as Java or C#, but reuse would be needed in languages such as C++ or C.

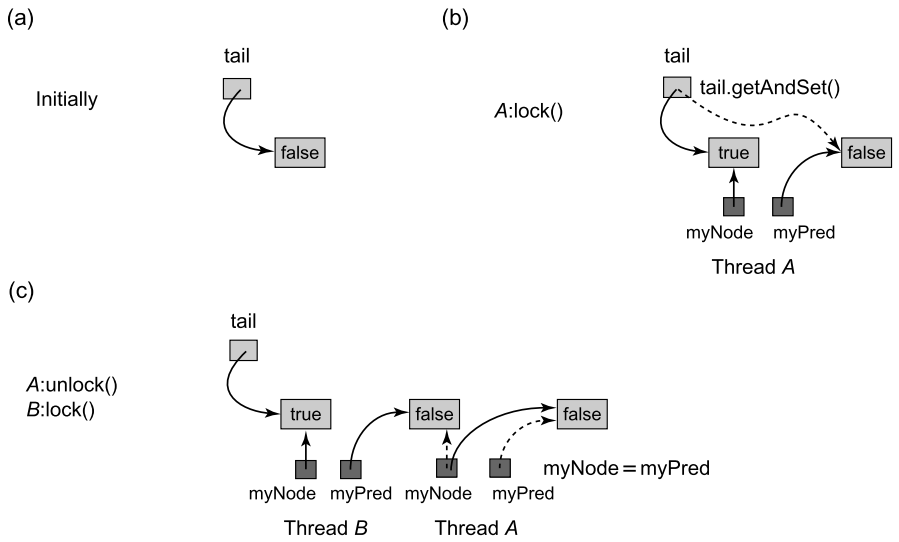

FIGURE 7.10

`CLHLock` class: lock acquisition and release. Initially the `tail` field refers to a `QNode` whose `locked` field is *false*. Thread *A* then applies `getAndSet()` to the `tail` field to insert its `QNode` at the tail of the queue, simultaneously acquiring a reference to its predecessor's `QNode`. Next, *B* does the same to insert its `QNode` at the tail of the queue. *A* then releases the lock by setting its node's `locked` field to *false*. It then recycles the `QNode` referenced by `pred` for future lock accesses.

the number of threads that might access the lock. Like the `ALock` class, it provides first-come-first-served fairness.

Perhaps the only disadvantage of this lock algorithm is that it performs poorly on cacheless NUMA architectures. Each thread spins waiting for its predecessor's node's `locked` field to become *false*. If this memory location is remote, then performance suffers. On cache-coherent architectures, however, this approach should work well.

7.5.3 The MCS queue lock

The `MCSLock` (Fig. 7.11) is another lock represented as a linked list of `QNode` objects, where each `QNode` represents either a lock holder or a thread waiting to acquire the lock. Unlike the `CLHLock` class, the list is explicit, not virtual: Instead of embodying the list in thread-local variables, it is embodied in the (globally accessible) `QNode` objects, via their `next` fields.

To acquire the lock, a thread appends its own `QNode` at the tail of the list (line 14). If the queue was not previously empty, it sets the predecessor's `QNode`'s `next` field to refer to its own `QNode`. The thread then spins on a (local) `locked` field in its own `QNode` waiting until its predecessor sets this field to *false* (lines 15–20).

```
1  public class MCSLock implements Lock {
2    AtomicReference<QNode> tail;
3    ThreadLocal<QNode> myNode;
4    public MCSLock() {
5      tail = new AtomicReference<QNode>(null);
6      myNode = new ThreadLocal<QNode>() {
7        protected QNode initialValue() {
8          return new QNode();
9        }
10     };
11   }
12   public void lock() {
13     QNode qnode = myNode.get();
14     QNode pred = tail.getAndSet(qnode);
15     if (pred != null) {
16       qnode.locked = true;
17       pred.next = qnode;
18       // wait until predecessor gives up the lock
19       while (qnode.locked) {}
20     }
21   }
22   public void unlock() {
23     QNode qnode = myNode.get();
24     if (qnode.next == null) {
25       if (tail.compareAndSet(qnode, null))
26         return;
27       // wait until successor fills in its next field
28       while (qnode.next == null) {}
29     }
30     qnode.next.locked = false;
31     qnode.next = null;
32   }
33   class QNode {
34     volatile boolean locked = false;
35     volatile QNode next = null;
36   }
37 }
```

FIGURE 7.11

The `MCSLock` class.

To release the lock, a thread checks whether its node's `next` field is *null* (line 24). If so, then either no other thread is contending for the lock, or there is another thread, but it is slow. To distinguish these cases, it applies `compareAndSet`(q, *null*) to the `tail` field, where q is the thread's node. If the call succeeds, then no other thread is trying to acquire the lock, so the thread just returns. Otherwise, another (slow) thread is

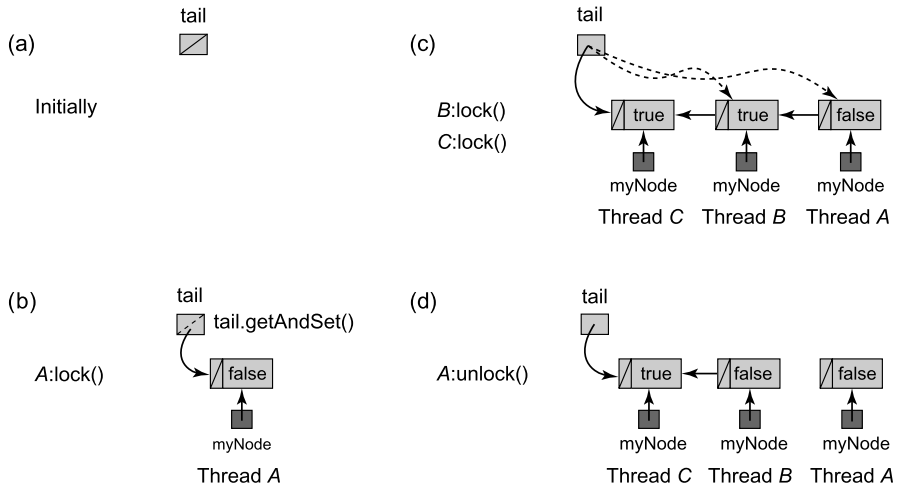

FIGURE 7.12

A lock acquisition and release in an `MCSLock`. (a) Initially the `tail` is *null*. (b) To acquire the lock, thread *A* places its own `QNode` at the tail of the list and since it has no predecessor, it enters the critical section. (c) Thread *B* enqueues its own `QNode` at the tail of the list and modifies its predecessor's `QNode` to refer back to its own. Thread *B* then spins on its `locked` field waiting until *A*, its predecessor, sets this field from *true* to *false*. Thread *C* repeats this sequence. (d) To release the lock, *A* follows its `next` field to its successor *B* and sets *B*'s `locked` field to *false*. It can now reuse its `QNode`.

trying to acquire the lock, so the thread spins waiting for the other thread to finish adding its node to the queue (line 28). Once the successor appears (or if it was there at the beginning), the thread sets its successor's `locked` field to *false*, indicating that the lock is now free. At this point, no other thread can access this `QNode`, and so it can be reused. Fig. 7.12 shows an example execution of the `MCSLock`.

This lock shares the advantages of the `CLHLock`, in particular, the property that each lock release invalidates only the successor's cache entry. It is better suited to cacheless NUMA architectures because each thread controls the location on which it spins. Like the `CLHLock`, nodes can be recycled so that this lock has space complexity $O(L + n)$. One drawback of the `MCSLock` algorithm is that releasing a lock requires spinning. Another is that it requires more reads, writes, and `compareAndSet()` calls than the `CLHLock` algorithm.

7.6 A queue lock with timeouts

The Java `Lock` interface includes a `tryLock()` method that allows the caller to specify a *timeout*, that is, a maximum duration the caller is willing to wait to acquire the lock. If the timeout expires before the caller acquires the lock, the attempt is aban-

```
1  public class TOLock implements Lock{
2    static QNode AVAILABLE = new QNode();
3    AtomicReference<QNode> tail;
4    ThreadLocal<QNode> myNode;
5    public TOLock() {
6      tail = new AtomicReference<QNode>(null);
7      myNode = new ThreadLocal<QNode>() {
8        protected QNode initialValue() {
9          return new QNode();
10       }
11     };
12   }
13   ...
14   static class QNode {
15     public volatile QNode pred = null;
16   }
17 }
```

FIGURE 7.13

`TOLock` class: fields, constructor, and `QNode` class.

doned. A Boolean return value indicates whether the lock attempt succeeded. (For an explanation why these methods throw `InterruptedException`, see Pragma 8.2.1.)

Abandoning a `BackoffLock` request is trivial: a thread can simply return from the `tryLock()` call. Responding to a timeout is wait-free, requiring only a constant number of steps. By contrast, timing out any of the queue lock algorithms is far from trivial: if a thread simply returns, the threads queued up behind it will starve.

Here is a bird's-eye view of a queue lock with timeouts. As in the `CLHLock`, the lock is a virtual queue of nodes, and each thread spins on its predecessor's node waiting for the lock to be released. As noted, when a thread times out, it cannot simply abandon its queue node, because its successor will never notice when the lock is released. On the other hand, it seems extremely difficult to unlink a queue node without disrupting concurrent lock releases. Instead, we take a *lazy* approach: When a thread times out, it marks its node as abandoned. Its successor in the queue, if there is one, notices that the node on which it is spinning has been abandoned, and starts spinning on the abandoned node's predecessor. This approach has the added advantage that the successor can recycle the abandoned node.

Fig. 7.13 shows the fields, constructor, and `QNode` class for the `TOLock` (timeout lock) class, a queue lock based on the `CLHLock` class that supports wait-free timeout even for threads in the middle of the list of nodes waiting for the lock.

When a `QNode`'s `pred` field is *null*, the associated thread has either not acquired the lock or has released it. When a `QNode`'s `pred` field refers to the distinguished static `QNode AVAILABLE`, the associated thread has released the lock. Finally, if the `pred` field refers to some other `QNode`, the associated thread has abandoned the lock request, so

```
18  public boolean tryLock(long time, TimeUnit unit) throws InterruptedException {
19    long startTime = System.currentTimeMillis();
20    long patience = TimeUnit.MILLISECONDS.convert(time, unit);
21    QNode qnode = new QNode();
22    myNode.set(qnode);
23    qnode.pred = null;
24    QNode myPred = tail.getAndSet(qnode);
25    if (myPred == null || myPred.pred == AVAILABLE) {
26      return true;
27    }
28    while (System.currentTimeMillis() - startTime < patience) {
29      QNode predPred = myPred.pred;
30      if (predPred == AVAILABLE) {
31        return true;
32      } else if (predPred != null) {
33        myPred = predPred;
34      }
35    }
36    if (!tail.compareAndSet(qnode, myPred))
37      qnode.pred = myPred;
38    return false;
39  }
40  public void unlock() {
41    QNode qnode = myNode.get();
42    if (!tail.compareAndSet(qnode, null))
43      qnode.pred = AVAILABLE;
44  }
```

FIGURE 7.14

TOLock class: tryLock() and unlock() methods.

the thread owning the successor node should wait on the abandoned node's predecessor.

Fig. 7.14 shows the TOLock class's tryLock() and unlock() methods. The tryLock() method creates a new QNode with a *null* pred field and appends it to the list as in the CLHLock class (lines 21–24). If the lock was free (line 25), the thread enters the critical section. Otherwise, it spins waiting for its predecessor's QNode's pred field to change (lines 28–35). If the predecessor thread times out, it sets the pred field to its own predecessor, and the thread spins instead on the new predecessor. An example of such a sequence appears in Fig. 7.15. Finally, if the thread itself times out (line 36), it attempts to remove its QNode from the list by applying compareAndSet() to the tail field. If the compareAndSet() call fails, indicating that the thread has a successor, the thread sets its QNode's pred field, previously *null*, to its predecessor's QNode, indicating that it has abandoned the queue.

In the unlock() method, a thread uses compareAndSet() to check whether it has a successor (line 42), and if so, sets its pred field to AVAILABLE. Note that it is not safe

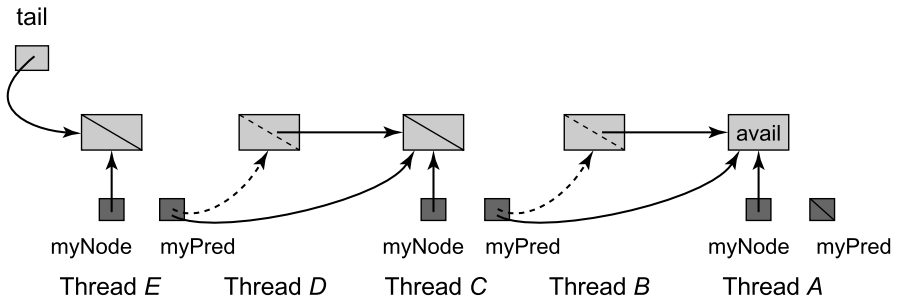

FIGURE 7.15

Timed-out nodes that must be skipped to acquire the `TOLock`. Threads *B* and *D* have timed out, redirecting their `pred` fields to their predecessors in the list. Thread *C* notices that *B*'s field is directed at *A* and so it starts spinning on *A*. Similarly, thread *E* spins waiting for *C*. When *A* completes and sets its `pred` to `AVAILABLE`, *C* will access the critical section and upon leaving it will set its `pred` to `AVAILABLE`, releasing *E*.

to recycle a thread's old node at this point, since the node may be referenced by its immediate successor, or by a chain of such references. The nodes in such a chain can be recycled as soon as a thread skips over the timed-out nodes and enters the critical section.

The `TOLock` has many of the advantages of the original `CLHLock`: local spinning on a cached location and quick detection that the lock is free. It also has the wait-free timeout property of the `BackoffLock`. However, it has some drawbacks, among them the need to allocate a new node per lock access and the fact that a thread spinning on the lock may have to traverse a chain of timed-out nodes before it can access the critical section.

7.7 Hierarchical locks

Many of today's cache-coherent architectures organize processors into *clusters*, where communication within a cluster is significantly faster than communication between clusters. For example, a cluster might correspond to a group of processors that share memory through a fast interconnect, or to the threads running on a single core in a multicore architecture. Such systems are called *nonuniform memory access* (NUMA) systems. On a NUMA system, passing a lock between threads in different clusters (i.e., *remote* threads) incurs significantly more overhead than passing it between threads in the same cluster (i.e., *local* threads). This increased overhead is due not only to the increased cost of synchronization on the lock, but also to the cost of transferring the data protected by the lock. We can reduce this overhead by preferentially passing the lock to a local thread rather than a remote one (i.e., to a thread in the same cluster as the thread releasing the lock, rather than to one in a different cluster). Such a lock is called a *hierarchical lock*.

```
1 public class ClusterLocal<T> {
2   protected T initialValue();
3   T get();
4   void set(T value);
5 }
```

FIGURE 7.16

The ClusterLocal class.

We consider an architecture with a two-level memory hierarchy, consisting of clusters of processors, where processors in the same cluster communicate efficiently through a shared cache, and intercluster communication is much more expensive than intracluster communication. For architectures whose memory hierarchy has more than two levels, we can apply the techniques in this section at each boundary between levels in the hierarchy.

We assume that each cluster has a unique *cluster ID* known to each thread in the cluster, available via ThreadID.getCluster(), and that threads do not migrate between clusters. We also assume there is a class `ClusterLocal<T>` (Fig. 7.16), analogous to `ThreadLocal<T>`, which manages one variable for each cluster, and provides `get()`, `set()`, and `initialValue()` for reading, writing, and initializing these variables.

7.7.1 A hierarchical back-off lock

Simple back-off locks, such as test-and-set and test-and-test-and-set locks, can easily be adapted to exploit clustering: By increasing the back-off times of threads in different clusters from the thread holding the lock (relative to those of threads in the same cluster), local threads are more likely to acquire the lock than remote threads. To do this, we must record the cluster of the thread that holds the lock. Fig. 7.17 shows the `HBOLock` class, a hierarchical back-off lock based on this principle.

`HBOLock` suffers from some of the same problems as `BackoffLock`, as described in Section 7.4. These problems may be even worse on NUMA systems because of the greater disparity in communication costs and the longer back-off times for remote threads. For example, longer back-off times increase delays between the release of a lock and its subsequent acquisition, resulting in greater underutilization of the critical section. As before, choosing back-off durations can be difficult, and acquiring or releasing the lock can generate a "storm" of cache-coherence traffic. And as with `BackoffLock`, the `HBOLock` may be *too* successful at passing the lock among threads in a single cluster, starving remote threads attempting to acquire the lock. In short, the problems with back-off locks that led us to explore queue locks still exist and are more severe on NUMA systems.

7.7.2 Cohort locks

We can address these problems by *lock cohorting*, a simple but effective technique that enables threads in a cluster to pass the lock among themselves without inter-

```
1  public class HBOLock implements Lock {
2    private static final int LOCAL_MIN_DELAY = ...;
3    private static final int LOCAL_MAX_DELAY = ...;
4    private static final int REMOTE_MIN_DELAY = ...;
5    private static final int REMOTE_MAX_DELAY = ...;
6    private static final int FREE = -1;
7    AtomicInteger state;
8    public HBOLock() {
9      state = new AtomicInteger(FREE);
10   }
11   public void lock() {
12     int myCluster = ThreadID.getCluster();
13     Backoff localBackoff =
14         new Backoff(LOCAL_MIN_DELAY, LOCAL_MAX_DELAY);
15     Backoff remoteBackoff =
16         new Backoff(REMOTE_MIN_DELAY, REMOTE_MAX_DELAY);
17     while (true) {
18       if (state.compareAndSet(FREE, myCluster)) {
19         return;
20       }
21       int lockState = state.get();
22       if (lockState == myCluster) {
23         localBackoff.backoff();
24       } else {
25         remoteBackoff.backoff();
26       }
27     }
28   }
29   public void unlock() {
30     state.set(FREE);
31   }
32 }
```

FIGURE 7.17

The `HBOLock` class: a hierarchical back-off lock.

cluster communication. The set of threads in a single cluster waiting to acquire the lock is called a *cohort*, and a lock based on this technique is called a *cohort lock*.

The key idea of lock cohorting is to use multiple locks to provide exclusion at different levels of the memory hierarchy. In a cohort lock, each cluster has a *cluster lock*, held by a thread, and the clusters share a *global lock*, held by a cluster. A thread holds the cohort lock if it holds its cluster lock and its cluster holds the global lock. To acquire the cohort lock, a thread first acquires the lock of its cluster, and then ensures that its cluster holds the global lock. When releasing the cohort lock, the thread checks whether there is any thread in its cohort (i.e., a thread in its cluster is waiting to acquire the lock). If so, the thread releases its cluster lock without releasing

```
1 public interface CohortDetectionLock extends Lock {
2   public boolean alone();
3 }
```

FIGURE 7.18

Interface for locks that support cohort detection.

the global lock. In this way, the thread in its cluster that next acquires the cluster lock also acquires the cohort lock (since its cluster already holds the global lock) without intercluster communication. If the cohort is empty when a thread releases the lock, it releases both the cluster lock and the global lock. To prevent remote threads from starving, a cohort lock must also have some policy that restricts local threads from passing the lock among themselves indefinitely without releasing the global lock.

A cohort lock algorithm requires certain properties of its component locks. A thread releasing the lock must be able to detect whether another thread is attempting to acquire its cluster lock, and it must be able to pass ownership of the global lock directly to another thread without releasing it.

A lock *supports cohort detection* if it provides a predicate method `alone()` with the following meaning: If `alone()` returns *false* when called by the thread holding a lock, then another thread is attempting to acquire that lock. The converse need not hold: If `alone()` returns *true*, there may be another thread attempting to acquire the lock, but such false positives should be rare. Fig. 7.18 shows an interface for a lock that supports cohort detection.

A lock is *thread-oblivious* if the thread releasing a thread-oblivious lock need not be the thread that most recently acquired it. The pattern of lock accesses must still be well formed (for example, `unlock()` may not be invoked when the lock is free).

Fig. 7.19 shows code for the `CohortLock` class, which must be instantiated with a thread-oblivious global lock and a lock that supports cohort detection for each cluster. The global lock must be thread-oblivious because its ownership may be passed implicitly among threads in a cluster, and eventually released by a different thread than the one that acquired the lock.

The `lock()` function acquires the thread's cluster lock, and then checks whether the lock was passed locally, meaning that its cluster already owns the global lock. If so, it returns immediately. Otherwise, it acquires the global lock before returning.

The `unlock()` function first determines whether a local thread is trying to acquire the lock, and if so, whether it should pass the lock locally. The latter decision is made by a "turn arbiter." We adopt a simple policy of bounding the number of times a thread may be passed locally without releasing the global lock. To emphasize that other policies are possible, we encapsulate the policy in a `TurnArbiter` class, shown in Fig. 7.20. The `passedLocally` field and the arbiter are updated to reflect the decision of whether to pass the lock locally. If the lock is not to be passed locally, both the global lock and the cluster lock are released. Otherwise, only the cluster lock is released.

```
1  public class CohortLock implements Lock {
2    final Lock globalLock;
3    final ClusterLocal<CohortDetectionLock> clusterLock;
4    final TurnArbiter localPassArbiter;
5    ClusterLocal<Boolean> passedLocally;
6    public CohortLock(Lock gl, ClusterLocal<CohortDetectonLock> cl, int passLimit) {
7      globalLock = gl;
8      clusterLock = cl;
9      localPassArbiter = new TurnArbiter(passLimit);
10   }
11   public void lock() {
12     clusterLock.get().lock();
13     if (passedLocally.get()) return;
14     globalLock.lock();
15   }
16   public void unlock() {
17     CohortDetectionLock cl = clusterLock.get();
18     if (cl.alone() || !localPassArbiter.goAgain()) {
19       localPassArbiter.passed();
20       passedLocally.set(false);
21       globalLock.unlock();
22     } else {
23       localPassArbiter.wentAgain();
24       passedLocally.set(true);
25     }
26     cl.unlock();
27   }
28 }
```

FIGURE 7.19

The `CohortLock` class.

7.7.3 A cohort lock implementation

We now describe a cohort lock implementation that uses `BackoffLock`, which is thread-oblivious, for the global lock, and a version of `MCSLock` modified to provide an `alone()` method for the cluster locks. The modified `MCSLock` is shown in Fig. 7.21. The `alone()` method simply checks whether the `next` field of the invoking thread's `QNode` is *null*. This test provides cohort detection, because whenever the `next` field of a `QNode` is not *null*, it points to the `QNode` of a thread waiting to acquire the lock. Fig. 7.22 shows how to extend `CohortLock` to use `BackoffLock` and the modified `MCSLock`. Fig. 7.23 illustrates an execution of this cohort lock.

`CohortBackoffMCSLock` can be improved slightly by recording in the `QNode` whether the lock has been passed locally. Instead of a `locked` field, the `QNode` maintains a field that indicates whether its thread must wait, or whether it has acquired the lock, and if so, whether the lock was passed locally or globally. There is no need for a separate cluster-local field to record whether the lock was passed locally, and the cache miss that would be incurred by accessing that field after the lock is acquired. We leave the details as an exercise.

```
1  public class TurnArbiter {
2    private final int TURN_LIMIT;
3    private int turns = 0;
4    public LocalPassingArbiter(int limit) {
5      TURN_LIMIT = limit;
6    }
7    public boolean goAgain() {
8      return (turns < TURN_LIMIT);
9    }
10   public void wentAgain() {
11     turns++;
12   }
13   public void passed() {
14     turns = 0;
15   }
16 }
```

FIGURE 7.20

`TurnArbiter` class.

```
1  public class CohortDetectionMCSLock extends MCSLock
2                    implements CohortDetectionLock {
3    public boolean alone() {
4      return (myNode.get().next == null);
5    }
6  }
```

FIGURE 7.21

Adding support for cohort detection to `MCSLock`.

```
1  public class CohortBackoffMCSLock extends CohortLock {
2    public CohortBackoffMCSLock(int passLimit) {
3      ClusterLocal<CohortDetectionMCSLock> cl = new ClusterLocal<CohortDetectionMCSLock> {
4        protected CohortDetectionMCSLock initialValue() {
5          return new CohortDetectionMCSLock();
6        }
7      }
8      super(new BackoffLock(), cl, passLimit);
9    }
10 }
```

FIGURE 7.22

`CohortBackoffMCSLock` class.

7.8 A composite lock

Spin lock algorithms impose trade-offs. Queue locks provide first-come-first-served fairness, fast lock release, and low contention, but require nontrivial protocols for

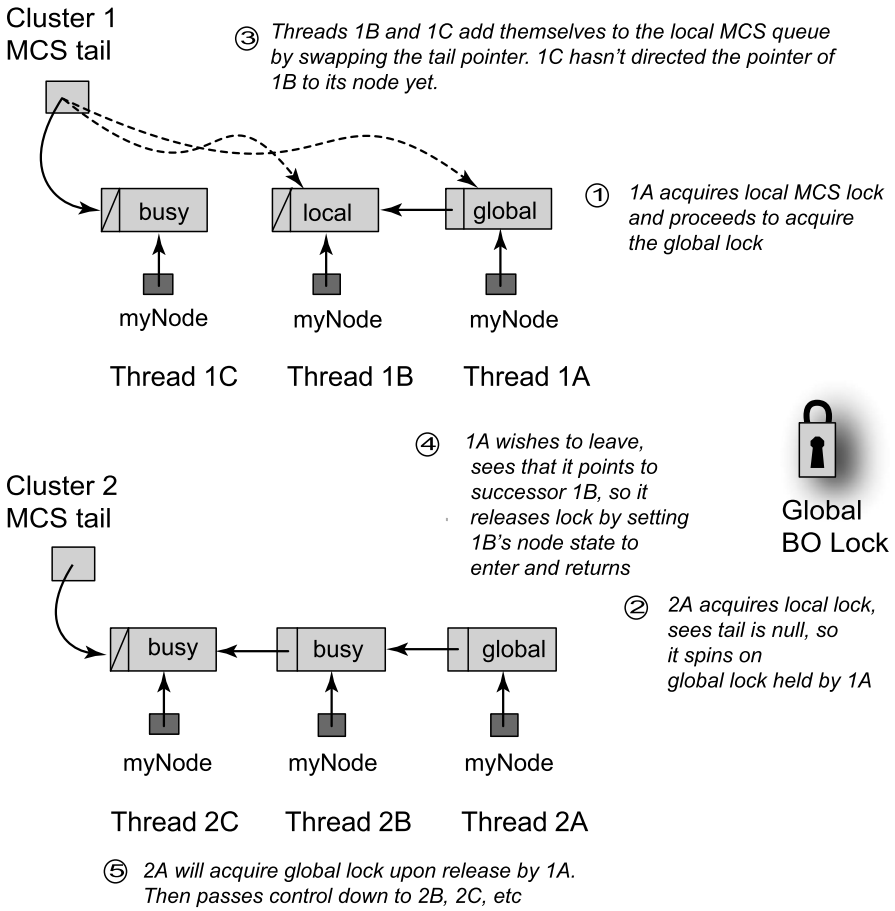

FIGURE 7.23

An example execution of `CohortBackoffMCSLock`.

recycling abandoned nodes. By contrast, back-off locks support trivial timeout protocols, but are inherently not scalable, and may have slow lock release if timeout parameters are not well tuned. In this section, we consider an advanced lock algorithm that combines the best of both approaches.

Consider the following simple observation: In a queue lock, only the threads at the front of the queue need to perform lock hand-offs. One way to balance the merits of queue locks versus back-off locks is to keep a small number of waiting threads in a queue on the way to the critical section, and have the rest use exponential back-off while attempting to enter this short queue. It is trivial for the threads employing back-off to quit.

```
1  public class CompositeLock implements Lock{
2    private static final int SIZE = ...;
3    private static final int MIN_BACKOFF = ...;
4    private static final int MAX_BACKOFF = ...;
5    AtomicStampedReference<QNode> tail;
6    QNode[] waiting;
7    ThreadLocal<QNode> myNode = new ThreadLocal<QNode>() {
8      protected QNode initialValue() { return null; };
9    };
10   public CompositeLock() {
11     tail = new AtomicStampedReference<QNode>(null,0);
12     waiting = new QNode[SIZE];
13     for (int i = 0; i < waiting.length; i++) {
14       waiting[i] = new QNode();
15     }
16   }
17   public void unlock() {
18     QNode acqNode = myNode.get();
19     acqNode.state.set(State.RELEASED);
20     myNode.set(null);
21   }
22   ...
23 }
```

FIGURE 7.24

The `CompositeLock` class: fields, constructor, and `unlock()` method.

The `CompositeLock` class keeps a short, fixed-size array of lock nodes. Each thread that tries to acquire the lock selects a node in the array at random. If that node is in use, the thread backs off (adaptively) and tries again. Once the thread acquires a node, it enqueues that node in a `TOLock`-style queue. The thread spins on the preceding node; when that node's owner signals it is done, the thread enters the critical section. When the thread leaves (after it completes or times out), it releases its node, and another thread may acquire it. The tricky part is recycling the freed nodes of the array while multiple threads attempt to acquire control over them.

The `CompositeLock`'s fields, constructor, and `unlock()` method appears in Fig. 7.24. The `tail` field is an `AtomicStampedReference<QNode>` that combines a reference to a node with a version number (see Pragma 10.6.1 for a more detailed explanation of the `AtomicStampedReference<T>` class); the version number is needed to avoid the *ABA* problem.[4] The `tail` field either is *null* or refers to the last node inserted in the queue.

[4] The ABA problem typically arises when using dynamically allocated memory in non-garbage-collected languages. See Section 10.6 for a more complete discussion of this problem in that context. We encounter it here because we are manually managing memory by using an array to implement a dynamic linked list.

```
24  enum State {FREE, WAITING, RELEASED, ABORTED};
25  class QNode {
26    AtomicReference<State> state;
27    QNode pred;
28    public QNode() {
29      state = new AtomicReference<State>(State.FREE);
30    }
31  }
```

FIGURE 7.25

The `CompositeLock` class: the `QNode` class.

```
32  public boolean tryLock(long time, TimeUnit unit) throws InterruptedException {
33    long patience = TimeUnit.MILLISECONDS.convert(time, unit);
34    long startTime = System.currentTimeMillis();
35    Backoff backoff = new Backoff(MIN_BACKOFF, MAX_BACKOFF);
36    try {
37      QNode node = acquireQNode(backoff, startTime, patience);
38      QNode pred = spliceQNode(node, startTime, patience);
39      waitForPredecessor(pred, node, startTime, patience);
40      return true;
41    } catch (TimeoutException e) {
42      return false;
43    }
44  }
```

FIGURE 7.26

The `CompositeLock` class: the `tryLock()` method.

Fig. 7.25 shows the `QNode` class. Each `QNode` includes a `State` field and a reference to the predecessor node in the queue. The `waiting` field is a constant-size `QNode` array.

A `QNode` has four possible states: `WAITING`, `RELEASED`, `ABORTED`, and `FREE`. A `WAITING` node is linked into the queue, and the owning thread is either in the critical section or waiting to enter. A node becomes `RELEASED` when its owner leaves the critical section and releases the lock. The other two states occur when a thread abandons its attempt to acquire the lock. If the quitting thread has acquired a node but not enqueued it, then it marks the thread as `FREE`. If the node is enqueued, then it is marked as `ABORTED`.

Fig. 7.26 shows the `tryLock()` method. A thread acquires the lock in three steps. It first *acquires* a node in the `waiting` array (line 37), then enqueues that node in the queue (line 38), and finally waits until that node is at the head of the queue (line 39).

The algorithm for acquiring a node in the `waiting` array appears in Fig. 7.27. The thread selects a node at random and tries to acquire the node by changing that node's state from `FREE` to `WAITING` (line 51). If it fails, it examines the node's status. If the node is `ABORTED` or `RELEASED` (line 56), the thread may "clean up" the node. To avoid synchronization conflicts with other threads, a node can be cleaned up only if it is

```
45   private QNode acquireQNode(Backoff backoff, long startTime, long patience)
46       throws TimeoutException, InterruptedException {
47     QNode node = waiting[ThreadLocalRandom.current().nextInt(SIZE)];
48     QNode currTail;
49     int[] currStamp = {0};
50     while (true) {
51       if (node.state.compareAndSet(State.FREE, State.WAITING)) {
52         return node;
53       }
54       currTail = tail.get(currStamp);
55       State state = node.state.get();
56       if (state == State.ABORTED || state == State.RELEASED) {
57         if (node == currTail) {
58           QNode myPred = null;
59           if (state == State.ABORTED) {
60             myPred = node.pred;
61           }
62           if (tail.compareAndSet(currTail, myPred, currStamp[0], currStamp[0]+1)) {
63             node.state.set(State.WAITING);
64             return node;
65           }
66         }
67       }
68       backoff.backoff();
69       if (timeout(patience, startTime)) {
70         throw new TimeoutException();
71       }
72     }
73   }
```

FIGURE 7.27

The `CompositeLock` class: the `acquireQNode()` method.

the last queue node (that is, the value of `tail`). If the tail node is `ABORTED`, `tail` is redirected to that node's predecessor; otherwise `tail` is set to *null*. If, instead, the allocated node is `WAITING`, then the thread backs off and retries. If the thread times out before acquiring its node, it throws `TimeoutException` (line 70).

Once the thread acquires a node, the `spliceQNode()` method (Fig. 7.28) splices that node into the queue by repeatedly trying to set `tail` to the allocated node. If it times out, it marks the allocated node as `FREE` and throws `TimeoutException`. If it succeeds, it returns the prior value of `tail`, acquired by the node's predecessor in the queue.

Finally, once the node has been enqueued, the thread must wait its turn by calling `waitForPredecessor()` (Fig. 7.29). If the predecessor is *null*, then the thread's node is first in the queue, so the thread saves the node in the thread-local `myNode` field (for later use by `unlock()`), and enters the critical section. If the predecessor node is not

```
74  private QNode spliceQNode(QNode node, long startTime, long patience)
75      throws TimeoutException {
76    QNode currTail;
77    int[] currStamp = {0};
78    do {
79      currTail = tail.get(currStamp);
80      if (timeout(startTime, patience)) {
81        node.state.set(State.FREE);
82        throw new TimeoutException();
83      }
84    } while (!tail.compareAndSet(currTail, node, currStamp[0], currStamp[0]+1));
85    return currTail;
86  }
```

FIGURE 7.28

The `CompositeLock` class: the `spliceQNode()` method.

```
87  private void waitForPredecessor(QNode pred, QNode node,
88                                  long startTime, long patience)
89      throws TimeoutException {
90    int[] stamp = {0};
91    if (pred == null) {
92      myNode.set(node);
93      return;
94    }
95    State predState = pred.state.get();
96    while (predState != State.RELEASED) {
97      if (predState == State.ABORTED) {
98        QNode temp = pred;
99        pred = pred.pred;
100       temp.state.set(State.FREE);
101     }
102     if (timeout(patience, startTime)) {
103       node.pred = pred;
104       node.state.set(State.ABORTED);
105       throw new TimeoutException();
106     }
107     predState = pred.state.get();
108   }
109   pred.state.set(State.FREE);
110   myNode.set(node);
111   return;
112 }
```

FIGURE 7.29

The `CompositeLock` class: the `waitForPredecessor()` method.

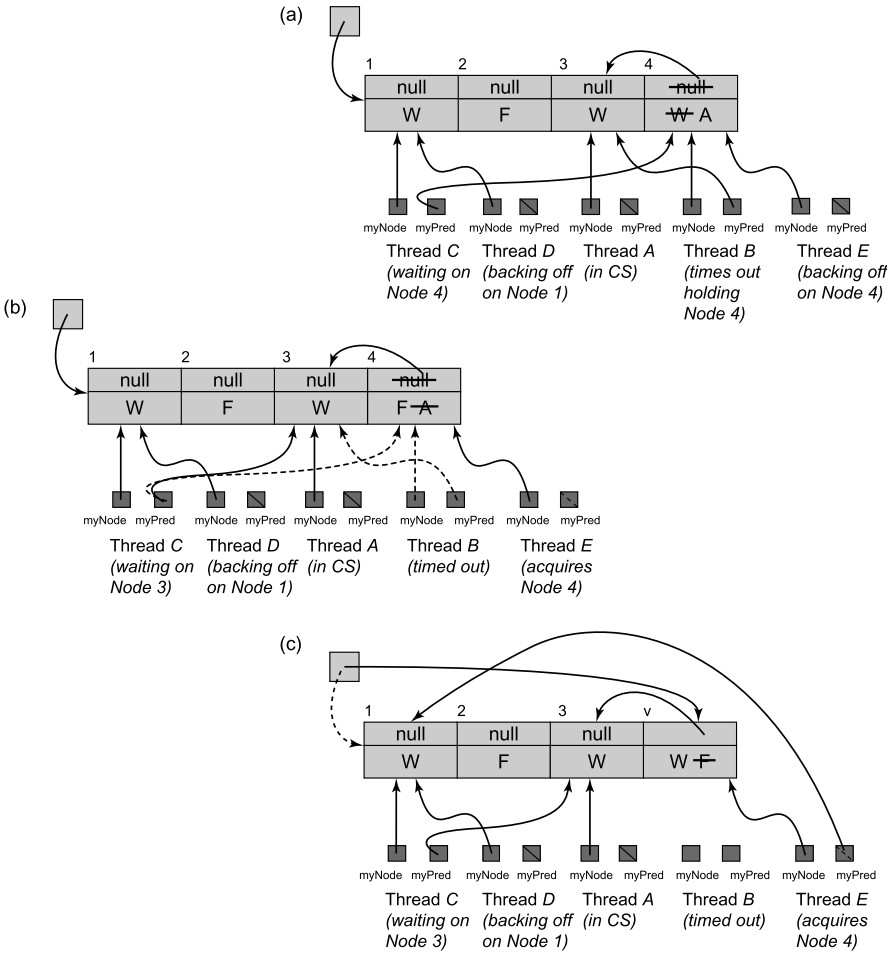

FIGURE 7.30

The `CompositeLock` class: an execution. In part (a), thread *A* (which acquired Node 3) is in the critical section. Thread *B* (Node 4) is waiting for *A* to release the critical section, and thread *C* (Node 1) is in turn waiting for *B*. Threads *D* and *E* are backing off, waiting to acquire a node. Node 2 is free. The `tail` field refers to Node 1, the last node to be inserted into the queue. At this point, *B* times out, inserting an explicit reference to its predecessor, and changing Node 4's state from `WAITING` (denoted by *W*) to `ABORTED` (denoted by *A*). In part (b), thread *C* cleans up the `ABORTED` Node 4, setting its state to `FREE` and following the explicit reference from 4 to 3 (by redirecting its local `myPred` field). It then starts waiting for *A* (Node 3) to leave the critical section. In part (c), *E* acquires the `FREE` Node 4, using `compareAndSet()` to set its state to `WAITING`. Thread *E* then inserts Node 4 into the queue, using `compareAndSet()` to swap Node 4 into the tail, then waiting on Node 1, which was previously referred to by `tail`.

RELEASED, the thread checks whether it is ABORTED (line 97). If so, the thread marks the node FREE and waits on the aborted node's predecessor. If the thread times out, then it marks its own node as ABORTED and throws TimeoutException. Otherwise, when the predecessor node becomes RELEASED the thread marks it FREE, records its own node in the thread-local myPred field, and enters the critical section.

The unlock() method (Fig. 7.24) simply retrieves its node from myNode and marks it RELEASED.

Fig. 7.30 illustrates an example execution of CompositeLock.

CompositeLock has a number of attractive properties. Lock hand-off is fast, just as in the CLHLock and TOLock algorithms. When threads back off, they access different locations, reducing contention. Abandoning a lock request is trivial for threads in the back-off stage, and relatively straightforward for threads that have acquired queue nodes. For L locks and n threads, the CompositeLock class requires only $O(L)$ space in the worst case, as compared to the TOLock class's $O(L \cdot n)$.

There are some drawbacks: CompositeLock does not guarantee first-come-first-served access. Also, a thread running alone must redirect the tail field away from a released node, claim the node, and then splice it into the queue.

7.9 A fast path for threads running alone

Although performance under contention is important, so is performance in the absence of concurrency. Ideally, for a thread running alone, acquiring a lock should be as simple as acquiring an uncontended TASLock. Unfortunately, as mentioned above, this is not true for the CompositeLock. We can address this shortcoming by adding a "fast path" to CompositeLock.

A *fast path* for a complex, expensive algorithm is a simpler, cheaper alternative that works (or is efficient) only under certain (typically, common) conditions. In this case, we want a fast path for CompositeLock for a thread that is running alone. We can accomplish this by extending the CompositeLock algorithm so that a solitary thread acquires an idle lock without acquiring a node and splicing it into the queue.

Here is a bird's-eye view. We add an extra state, distinguishing between a lock held by an ordinary thread and a lock held by a fast-path thread. If a thread discovers the lock is free, it tries a fast-path acquire. If it succeeds, then it has acquired the lock in a single atomic step. If it fails, then it enqueues itself just as before.

We now examine the algorithm in detail. To reduce code duplication, we define the CompositeFastPathLock class to be a subclass of CompositeLock. The code appears in Figs. 7.31 and 7.32.

We use a *fast-path flag* to indicate that a thread has acquired the lock through the fast path. Because we need to manipulate this flag together with the tail field's reference, we "steal" a high-order bit from the tail field's integer stamp using a FASTPATH bitmask (line 2). The private fastPathLock() method checks whether the tail field's stamp has a clear fast-path flag and a *null* reference. If so, it tries to acquire the lock simply by applying compareAndSet() to set the fast-path flag to *true*, ensuring that

```
1  public class CompositeFastPathLock extends CompositeLock {
2    private static final int FASTPATH = 1 << 30;
3    private boolean fastPathLock() {
4      int oldStamp, newStamp;
5      int stamp[] = {0};
6      QNode qnode;
7      qnode = tail.get(stamp);
8      oldStamp = stamp[0];
9      if (qnode != null) {
10       return false;
11     }
12     if ((oldStamp & FASTPATH) != 0) {
13       return false;
14     }
15     newStamp = (oldStamp + 1) | FASTPATH;
16     return tail.compareAndSet(qnode, null, oldStamp, newStamp);
17   }
18   public boolean tryLock(long time, TimeUnit unit) throws InterruptedException {
19     if (fastPathLock()) {
20       return true;
21     }
22     if (super.tryLock(time, unit)) {
23       while ((tail.getStamp() & FASTPATH ) != 0){};
24       return true;
25     }
26     return false;
27   }
```

FIGURE 7.31

`CompositeFastPathLock` class: The private `fastPathLock()` method returns true if it succeeds in acquiring the lock through the fast path.

the reference remains *null*. An uncontended lock acquisition thus requires a single atomic operation. The `fastPathLock()` method returns *true* if it succeeds, and *false* otherwise.

The `tryLock()` method (lines 18–27) first tries the fast path by calling `fastPathLock()`. If it fails, then it pursues the slow path by calling the `CompositeLock` class's `tryLock()` method. Before it can return from the slow path, however, it must ensure that no other thread holds the fast-path lock by waiting until the fast-path flag is clear (line 23).

The `unlock()` method first calls `fastPathUnlock()` (line 44). If that call fails to release the lock, it then calls the `CompositeLock`'s `unlock()` method (line 45). The `fastPathUnlock()` method returns *false* if the fast-path flag is not set (line 31). Otherwise, it repeatedly tries to clear the flag, leaving the reference component unchanged (lines 36–40), returning *true* when it succeeds.

```
28   private boolean fastPathUnlock() {
29     int oldStamp, newStamp;
30     oldStamp = tail.getStamp();
31     if ((oldStamp & FASTPATH) == 0) {
32       return false;
33     }
34     int[] stamp = {0};
35     QNode qnode;
36     do {
37       qnode = tail.get(stamp);
38       oldStamp = stamp[0];
39       newStamp = oldStamp & (~FASTPATH);
40     } while (!tail.compareAndSet(qnode, qnode, oldStamp, newStamp));
41     return true;
42   }
43   public void unlock() {
44     if (!fastPathUnlock()) {
45       super.unlock();
46     };
47   }
```

FIGURE 7.32

`CompositeFastPathLock` class: `fastPathUnlock()` and `unlock()` methods.

7.10 One lock to rule them all

In this chapter, we have seen a variety of spin locks that vary in characteristics and performance. Such a variety is useful, because no single algorithm is ideal for all applications. For some applications, complex algorithms work best; for others, simple algorithms are preferable. The best choice usually depends on specific aspects of the application and the target architecture.

7.11 Chapter notes

The `TTASLock` is due to Larry Rudolph and Zary Segall [150]. Exponential back-off is a well-known technique used in ethernet routing, presented in the context of multiprocessor mutual exclusion by Anant Agarwal and Mathews Cherian [6]. Tom Anderson [12] invented the `ALock` algorithm and was one of the first to empirically study the performance of spin locks in shared-memory multiprocessors. The `MCSLock`, due to John Mellor-Crummey and Michael Scott [124], is perhaps the best-known queue lock algorithm. Today's Java virtual machines use object synchronization based on simplified monitor algorithms such as the *Thinlock* of David Bacon, Ravi Konuru, Chet Murthy, and Mauricio Serrano [15], the *Metalock* of Ole Agesen, Dave Detlefs,

Alex Garthwaite, Ross Knippel, Y. S. Ramakrishna, and Derek White [7], or the *RelaxedLock* of Dave Dice [36]. All these algorithms are variations of the `MCSLock` lock.

The `CLHLock` lock is due to Travis Craig, Erik Hagersten, and Anders Landin [32, 118]. The `TOLock` with nonblocking timeout is due to Bill Scherer and Michael Scott [153,154]. The `CompositeLock` and its variations are due to Virendra Marathe, Mark Moir, and Nir Shavit [121]. The notion of using a fast path in a mutual exclusion algorithm is due to Leslie Lamport [106]. Hierarchical locks were invented by Zoran Radović and Erik Hagersten. The `HBOLock` is a variant of their original algorithm [144]. Cohort locks are due to Dave Dice, Virendra Marathe, and Nir Shavit [37].

Faith Fich, Danny Hendler, and Nir Shavit [45] have extended the work of Jim Burns and Nancy Lynch to show that any starvation-free mutual exclusion algorithm requires $\Omega(n)$ space, even if strong operations such as `getAndSet()` or `compareAndSet()` are used, implying that the queue-lock algorithms considered here are space-optimal.

The schematic performance graph in this chapter is loosely based on empirical studies by Tom Anderson [12], as well as on data collected by the authors on various modern machines. We present schematic rather than actual data because of the great variation in machine architectures and their significant effect on lock performance.

Programming languages such as C or C++ were not defined with concurrency in mind, so they did not define a memory model. The actual behavior of a concurrent C or C++ program is the result of a complex combination of the underlying hardware, the compiler, and the concurrency library. See Hans Boehm [19] for a more detailed discussion of these issues. The Java memory model proposed here is the *second* memory model proposed for Java. Jeremy Manson, Bill Pugh, and Sarita Adve [119] give a more complete description of this model.

The Sherlock Holmes quote is from *The Sign of Four* [41].

7.12 Exercises

Exercise 7.1. Fig. 7.33 shows an alternative implementation of `CLHLock` in which a thread reuses its own node instead of its predecessor node. Explain how this implementation can go wrong, and how the MCS lock avoids the problem even though it reuses thread-local nodes.

Exercise 7.2. Imagine n threads, each of which executes method `foo()` followed by method `bar()`. Suppose we want to make sure that no thread starts `bar()` until all threads have finished `foo()`. For this kind of synchronization, we place a *barrier* between `foo()` and `bar()`.

First barrier implementation: We have a counter protected by a test-and-test-and-set lock. Each thread locks the counter, increments it, releases the lock, and spins, rereading the counter until it reaches n.

Second barrier implementation: We have an n-element Boolean array `b`$[0..n-1]$, all initially *false*. Thread 0 sets `b`[0] to *true*. Every thread i, for $0 < i < n$, spins until

```
1  public class BadCLHLock implements Lock {
2    AtomicReference<Qnode> tail = new AtomicReference<QNode>(new QNode());
3    ThreadLocal<Qnode> myNode = new ThreadLocal<QNode> {
4      protected QNode initialValue() {
5        return new QNode();
6      }
7    };
8    public void lock() {
9      Qnode qnode = myNode.get();
10     qnode.locked = true;      // I'm not done
11     // Make me the new tail, and find my predecessor
12     Qnode pred = tail.getAndSet(qnode);
13     while (pred.locked) {}
14   }
15   public void unlock() {
16     // reuse my node next time
17     myNode.get().locked = false;
18   }
19   static class Qnode { // Queue node inner class
20     volatile boolean locked = false;
21   }
22 }
```

FIGURE 7.33

An incorrect attempt to implement a `CLHLock`.

`b`$[i-1]$ is *true*, sets `b`$[i]$ to *true*, and then waits until `b`$[n-1]$ is *true*, after which it proceeds to leave the barrier.

Compare (in 10 lines) the behavior of these two implementations on a bus-based cache-coherent architecture. Explain which approach you expect will perform better under low load and high load.

Exercise 7.3. Show how to eliminate the separate cluster-local field that records whether the lock is passed locally by recording this information directly in each `QNode`, as described in Section 7.7.3.

Exercise 7.4. Prove that the `CompositeFastPathLock` implementation guarantees mutual exclusion, but is not starvation-free.

Exercise 7.5. Design an `isLocked()` method that tests whether any thread is holding a lock (but does not acquire the lock). Give implementations for

- a test-and-set spin lock,
- the CLH queue lock, and
- the MCS queue lock.

Exercise 7.6. (Hard) Where does the $\Omega(n)$ space complexity lower bound proof for deadlock-free mutual exclusion of Chapter 2 break when locks are allowed to use read–modify–write operations?